Data Analytics (DA)

(Module-End -3) Module End Assignment

"Analyzing E-Learning Platform Purchases using MySQL"

Assignment Task Overview

Problem Statement:

An online learning platform sells various digital courses to learners across different countries. Each learner can purchase multiple courses, and each course belongs to a specific category.

The management team wants to analyze purchase data to understand **sales trends**, **learner behavior**, and **popular course categories**.

Your task is to use **MySQL** to design and query a relational database that helps derive meaningful insights from this e-learning purchase data.

Dataset Description:

You will work with the following three tables:

1. **Table: *learners***

   Attributes:

   - *learner_id (Primary Key)*
   - *full_name*
   - *Country*

2. **Table: *courses***

   Attributes:

   - *course_id (Primary Key)*
   - *course_name*
   - *category*
   - *unit_price*

3. **Table: *purchases***

Attributes:

- purchase_id (Primary Key)
- learner_id (Foreign Key → learners.learner_id)
- course_id (Foreign Key → courses.course_id)
- Quantity
- purchase_date

```sql
-- Database Creation --

Create Database if not exists elearning_purchases;
Use elearning_purchases;

-- Table Creation Learners, Courses, Purchases --

Create table learners (
    Learner_id  int Primary Key,
    Full_name varchar(100) not null,
    Country varchar(50) not null
);

Create table courses (
    Course_id int Primary Key,
    Course_name varchar(100) not null,
    Category varchar(50) not null,
    Unit_price decimal(10,2) not null
    );

Create table purchases(
    Purchase_id int Primary Key,
    Learner_id int,
    Course_id int,
    Quantity int,
    Purchase_date date not null,
    foreign key (learner_id) references learners(learner_id)
    on update cascade on delete cascade,
    foreign key (course_id) references courses(course_id)
    on update cascade on delete cascade
```

Tasks

1. Create the database and schema. Populate the Schema:

- Create a Database for this project and

Create all three tables in MySQL with appropriate **data types** and **relationships**.

- Insert **sample data** covering at least:
  - 4–5 learners
  - 4–5 courses (spread across multiple categories)
  - 6–8 purchase records

```
insert into learners values
(1,'Abi','India'),
(2,'Janani','USA'),
(3,'Soundarya','UAE'),
(4,'Aswathy','India'),
(5,'Mithran','USA');


insert into courses values
(101,'Data Analytics','data visulization',625.00),
(102,'Data Science','Machine Learning',785.98),
(103,'Digital Marketing','communication',525.75),
(104,'Machine Learning','AI', 650.38),
(105,'UI/UX','Designing',435.29);


insert into purchases(purchase_id,learner_id,course_id,quantity,purchase_date) values
(11,1,101,1,'2024-01-01'),
(12,2,102,2,'2024-02-01'),
(13,3,103,2,'2024-02-15'),
(14,4,104,4,'2024-02-01'),
(15,5,105,5,'2024-01-24'),
(16,3,103,3,'2024-01-15'),
(17,1,101,2,'2024-01-21');
```

2. Data Exploration Using Joins

Data Presentation Guidelines for the following query

- Format currency values to **2 decimal places**.
- Use **aliases** for column names (e.g., AS total_revenue).
- Sort results appropriately (e.g., highest total_spent first).

Use SQL INNER JOIN, LEFT JOIN, and RIGHT JOIN to:

- Combine learner, course, and purchase data.
- Display each learner's purchase details (course name, category, quantity, total amount, and purchase date).

```sql
select l.learner_id, c.course_name, p.quantity,
format(p.quantity*c.unit_price,2) as total_revenue,
p.purchase_date
from purchases p
inner join learners l on p.learner_id =l.learner_id
inner join courses c on p.course_id=c.course_id
order by total_revenue desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

learner_id	course_name	quantity	total_revenue	purchase_date
1	Data Analytics	1	625.00	2024-01-01
4	Machine Learning	4	2,601.52	2024-02-01
5	UI/UX	5	2,176.45	2024-01-24
3	Digital Marketing	3	1,577.25	2024-01-15
2	Data Science	2	1,571.96	2024-02-01
1	Data Analytics	2	1,250.00	2024-01-21
3	Digital Marketing	2	1,051.50	2024-02-15

```sql
SELECT
l.full_name,
c.course_name,
p.purchase_date
FROM learners l
LEFT JOIN purchases p ON l.learner_id = p.learner_id
LEFT JOIN courses c ON p.course_id = c.course_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

full_name	course_name	purchase_date
Abi	Data Analytics	2024-01-01
Abi	Data Analytics	2024-01-21
Janani	Data Science	2024-02-01
Soundarya	Digital Marketing	2024-02-15
Soundarya	Digital Marketing	2024-01-15
Aswathy	Machine Learning	2024-02-01
Mithran	UI/UX	2024-01-24

```sql
select l.full_name, c.course_name, p.quantity
from learners l
right join purchases p on l.learner_id=p.learner_id
right join courses c on c.course_id = p.course_id;

-- SQL Queries --
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conten

full_name	course_name	quantity
Abi	Data Analytics	1
Abi	Data Analytics	2
Janani	Data Science	2
Soundarya	Digital Marketing	2
Soundarya	Digital Marketing	3
Aswathy	Machine Learning	4
Mithran	UI/UX	5

3. Analytical Queries

Write SQL queries to answer the following questions:

Q1. Display each learner's total spending (quantity × unit_price) along with their country.

```
FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_spent
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
JOIN courses c ON p.course_id = c.course_id
GROUP BY l.full_name, l.country
ORDER BY total_spent DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	country	total_spent
Soundarya	UAE	2,628.75
Aswathy	India	2,601.52
Mithran	USA	2,176.45
Abi	India	1,875.00
Janani	USA	1,571.96

Q2. Find the top 3 most purchased courses based on total quantity sold.

```
select c.course_name,sum(p.quantity)as total_quantity_sold from purchases p
join courses c on c.course_id=p.course_id
group by c.course_name
order by total_quantity_sold desc
limit 3;

-- Q3. Show each course category's total revenue and the number of unique lear
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

course_name	total_quantity_sold
UI/UX	5
Digital Marketing	5
Machine Learning	4

Q3. Show each course category's total revenue and the number of unique learners who purchased from that category.

```
select c.category,
sum(p.quantity*c.unit_price) as total_revenue,
count(distinct l.learner_id) as unique_learner
from purchases p inner join courses c on c.course_id=p.course_id
inner join learners l on l.learner_id = p.learner_id
group by c.category
order by total_revenue desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	total_revenue	unique_learner
communication	2628.75	1
AI	2601.52	1
Designing	2176.45	1
data visulization	1875.00	1
Machine Learning	1571.96	1

Q4. List all learners who have purchased courses from more than one category.

```
select l.full_name,
count(c.category)as count_category
from learners l
inner join purchases p on l.learner_id = p.learner_id
inner  join courses c on c.course_id=p.course_id
group by l.full_name
having count_category>1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	count_category
Abi	2
Soundarya	2

Q5. Identify courses that have not been purchased at all.

```
select c.course_name,
c.category
from courses c left join purchases p on p.course_id =c.course_id
where p.purchase_id is null;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

course_name	category

Q6. Which country generates the highest total revenue

```
SELECT l.country,
FORMAT(SUM(p.quantity * c.unit_price), 2) AS total_revenue
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
JOIN courses c ON p.course_id = c.course_id
GROUP BY l.country
ORDER BY total_revenue DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country	total_revenue
India	4,476.52
USA	3,748.41
UAE	2,628.75

Q7. Find the least purchased course by total quantity code

```sql
SELECT c.course_name,
SUM(p.quantity) AS total_quantity_sold
FROM courses c
JOIN purchases p ON c.course_id = p.course_id
GROUP BY c.course_id, c.course_name
ORDER BY total_quantity_sold ASC LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

course_name	total_quantity_sold
Data Science	2

Q8. First Purchase Date of Each Learner

```sql
SELECT l.full_name AS learner_name,
MIN(p.purchase_date) AS first_purchase_date
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
GROUP BY l.learner_id
ORDER BY first_purchase_date;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

learner_name	first_purchase_date
Abi	2024-01-01
Soundarya	2024-01-15
Mithran	2024-01-24
Janani	2024-02-01
Aswathy	2024-02-01

Q9. Learners Who Made Only One Purchase

```sql
SELECT l.full_name AS learner_name,
COUNT(p.purchase_id) AS total_purchases
FROM learners l
JOIN purchases p ON l.learner_id = p.learner_id
GROUP BY l.learner_id
HAVING COUNT(p.purchase_id) = 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

learner_name	total_purchases
Janani	1
Aswathy	1
Mithran	1

Q10. Which category has the highest average course price

```sql
SELECT category,
FORMAT(AVG(unit_price), 2) AS avg_course_price
FROM courses
GROUP BY category
ORDER BY avg_course_price DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fe

category	avg_course_price
Machine Learning	785.98